Capitolo 7

Design pattern: principi e pattern creazionali

7.1 Design pattern: problemi ricorrenti, ruoli e conseguenze

Un *design pattern* è una micro-architettura: una struttura software, in genere formata da poche classi e oggetti, che descrive una soluzione collaudata, generica ma specializzabile, a un problema di progettazione ricorrente in un dato contesto. Non è una libreria né codice da copiare: identifica ruoli, responsabilità e collaborazioni che devono essere adattati al problema. Il suo scopo riguarda soprattutto le proprietà non funzionali di una soluzione—per esempio riuso, modificabilità, testabilità, affidabilità e sicurezza—oltre a ciò che il programma fa.

I pattern danno un vocabolario comune al gruppo di sviluppo e consentono di riusare soluzioni anziché reinventarle. Non sono però una garanzia automatica di qualità né vanno applicati senza un problema che ne giustifichi il costo: introducono classi, indirezioni e responsabilità ulteriori. Esistono cataloghi per contesti diversi (sistemi centralizzati, concorrenti, distribuiti e real-time); la progettazione orientata agli oggetti ne costituisce uno degli ambiti principali.

Per comprendere e applicare un pattern occorre precisarne cinque aspetti.

Nome identifica la soluzione e ne richiama lo scopo a un livello di astrazione alto.

Intento esprime brevemente ciò che il pattern vuole ottenere.

Problema comprende motivazione e applicabilità: precisa il contesto e le condizioni in cui la soluzione è appropriata.

Soluzione presenta classi, oggetti, responsabilità e relazioni; non è una singola implementazione rigida.

Conseguenze rendono espliciti risultati, vantaggi, svantaggi e compromessi.

Nel problema agiscono le *forze*, cioè obiettivi e vincoli spesso contrastanti che la progettazione deve bilanciare. Esempi di utilizzo e codice possono completare la descrizione.

I pattern si classificano secondo lo scopo: i *creazionali* organizzano la creazione di istanze; gli *strutturali* la composizione di classi e oggetti; i *comportamentali* gli algoritmi, la comunicazione e la distribuzione delle responsabilità. Questa classificazione distingue il problema affrontato, anche quando diagrammi di pattern diversi hanno una struttura simile.

Nel catalogo GoF i creazionali sono Abstract Factory, Builder, Factory Method, Prototype e Singleton; gli strutturali Adapter, Bridge, Composite, Decorator, Facade, Flyweight e Proxy; i comportamentali Chain of Responsibility, Command, Interpreter, Iterator, Mediator, Memento, Observer, State, Strategy, Template Method e Visitor. La classificazione colloca anche i nomi non sviluppati in dettaglio; Object Pool, Dependency Injection, MVC e Null

Object appartengono a schemi o tecniche ulteriori e non vanno conteggiati automaticamente nei 23 pattern del catalogo.

Una soluzione elementare può presupporre un sistema centralizzato e un singolo thread logico. Concorrenza, distribuzione e vincoli real-time introducono forze ulteriori: applicare un pattern in tali contesti richiede di riesaminarne le condizioni di funzionamento.

7.2 Quale aspetto della costruzione deve variare?

I pattern creazionali astraggono il processo di creazione: rendono il sistema meno dipendente da come gli oggetti sono creati, composti e rappresentati. Invece di disseminare espressioni `new ClasseConcreta(...)` nel codice client, spostano in un punto dedicato la conoscenza della classe concreta e della costruzione. Il *client* è il codice che richiede e usa un oggetto; una *classe concreta* è una classe istanziabile che realizza un'interfaccia o una classe astratta.

Il client riceve così un oggetto pronto e lo usa attraverso un'astrazione, cioè un'interfaccia o un tipo astratto. Questo favorisce il disaccoppiamento: una dipendenza da una concreta può essere sostituita senza modificare ogni client. Una gerarchia di ereditarietà è fissata in compilazione; una maggiore flessibilità può derivare dalla *composizione*, cioè della costruzione di un oggetto mediante riferimenti ad altri oggetti configurabili a runtime. Non significa che ogni `new` sia errato: l'istanziazione concreta resta necessaria da qualche parte; il punto è localizzarla quando la scelta o la costruzione devono variare.

Un vincolo di unicità conduce a Singleton; la variazione del prodotto a Factory Method; la coerenza di una famiglia ad Abstract Factory; la costruzione per passi a Builder; la creazione per copia a Prototype. Object Pool aggiunge una politica di riuso delle istanze, mentre Dependency Injection separa la costruzione dei collaboratori dal loro utilizzo.

7.3 Singleton: unicità e accesso controllato

L'intento di *Singleton* è assicurare che una classe abbia una sola istanza e fornire un punto di accesso globale a quell'istanza. È appropriato quando il dominio richiede effettivamente un solo gestore condiviso nell'applicazione, per esempio uno spooler di stampa, un file system, un window manager, un registro di log o il gestore della lista dei clienti. Una variabile globale rende un oggetto raggiungibile ma, da sola, non impedisce a qualcuno di costruire altre istanze: il vincolo deve essere controllato dalla classe.

La classe Singleton conserva un riferimento statico all'istanza, rende privato il costruttore e offre un metodo statico `getInstance()`. "Statico" qui significa che il membro appartiene alla classe, perciò `getInstance()` è invocabile prima che esista un oggetto. Il costruttore privato impedisce alle altre classi di usare `new`; il metodo della classe è l'unico punto autorizzato a farlo.

Con inizializzazione *eager*, l'istanza è creata durante l'inizializzazione della classe; con inizializzazione *lazy*, alla prima richiesta. La seconda evita di costruire un oggetto che potrebbe non essere usato, ma il controllo elementare sul riferimento nullo non è sicuro con più thread: due chiamanti possono superarlo contemporaneamente e creare due oggetti. L'inizializzazione della classe protegge invece la creazione eager; nella variante lazy un `getInstance()` sincronizzato può serializzare controllo e costruzione.

Il *Multiton* è la variante che consente un numero finito di istanze, eventualmente associandone una a una chiave, quale una cartella nel caso di `Pagamenti`. Il beneficio è il principio delle *conseguenze locali*: cambiando il numero ammesso di istanze si modifica il fornitore, mentre i client continuano a chiedere l'istanza con la stessa operazione. Rispetto a rendere tutte le operazioni statiche, Singleton conserva un oggetto e consente di variare centralmente la politica sul numero di oggetti; la soluzione interamente statica non crea invece alcuna istanza.

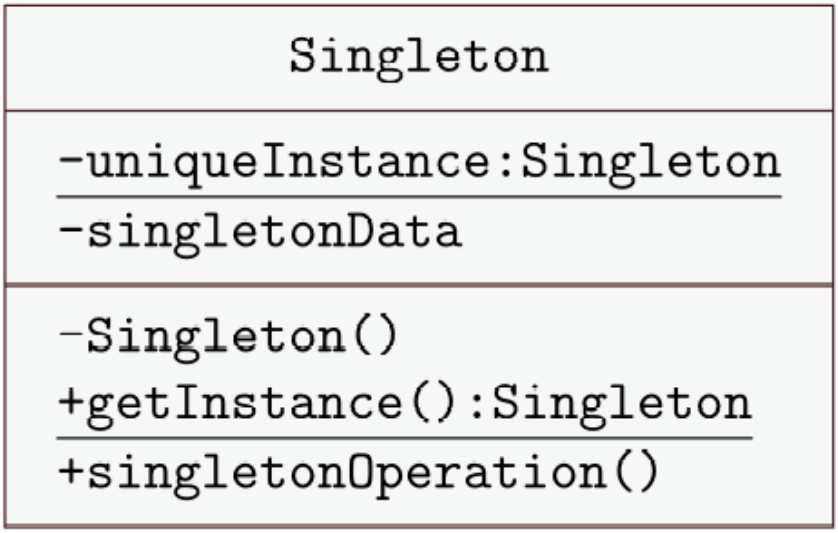


Figura 7.1: Costruttore privato, riferimento e accesso statici; operazioni sullo stato dell'unica istanza.

La sicurezza della creazione non rende automaticamente sicuri i metodi che modificano lo stato condiviso. Il passaggio a Multiton mantiene invariati i client soltanto se il contratto di accesso e le loro assunzioni sull'identità restano validi.

Il vincolo generale sul numero di istanze della classe C è

$$|I(C)| \leq 1, \tag{7.1}$$

dove $I(C)$ è l'insieme delle istanze. Per una creazione lazy può essere inizialmente vuoto; dopo l'inizializzazione riuscita ha un elemento. La sola visibilità globale non impone tale vincolo.

7.3.1 Logs lazy, Fib eager e variante Multiton

L'esempio `Logs` conserva i messaggi in una lista e crea l'unica istanza alla prima richiesta. La condizione sul riferimento evita una seconda costruzione nell'esecuzione a singolo thread considerata dall'esempio.

```
import java.util.List;
import java.util.ArrayList;

public final class Logs {
    private static Logs obj;
    private final List<String> righe = new ArrayList<>();
    private Logs() { }

    public static Logs getInstance() {
        if (obj == null) obj = new Logs();
        return obj;
    }
    public void record(String s) { righe.add(s); }
    public void empty() { righe.clear(); }
    public String dumpLast() { return righe.get(righe.size() - 1); }
    public String dumpAll() {
        String risultato = "";
        for (String s : righe) risultato += s;
```

```
        return risultato;
    }
}
```

`Logs a = Logs.getInstance()` e `Logs b = Logs.getInstance()` ricevono lo stesso riferimento. Questo codice presuppone un solo thread e richiede un adattamento per gestire richieste concorrenti.

`Fib`, che svolge il ruolo di Singleton per i numeri di Fibonacci, inizializza il riferimento statico direttamente alla dichiarazione: `private static Fib obj = new Fib();`. `Logs` rimanda invece la costruzione. Il nome del ruolo non è una keyword Java. Nel Multiton `Pagamenti` può associare una diversa istanza a ciascuna cartella; il criterio è se esista già un'istanza per la chiave richiesta.

`dumpLast` richiede almeno un messaggio; `dumpAll` concatena senza inserire separatori. Se si registrano `"one "`, `"two "`, `"three "`, l'ultimo messaggio è `"three "` e l'intero testo è `"one two three "`, con spazi finali significativi. Un test deve chiamare `empty` prima di preparare ciascuna fixture. Un secondo riferimento ottenuto da `getInstance()`, registrando poi `"uno"` e `"due"`, rende `"due"` visibile anche dal primo: l'identità condivisa può essere controllata con `assertSame(a,b)` e l'effetto con `assertEquals("due",a.dumpLast())`. Questi sono controlli distinti dell'unicità e dello stato.

I tre controlli si possono esprimere con una fixture indipendente per ogni test:

```
import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

class LogsTest {
    private final Logs l = Logs.getInstance();
    @BeforeEach void prepara() {
        l.empty();
        l.record("one "); l.record("two "); l.record("three ");
    }
    @Test void testLast() { assertEquals("three ", l.dumpLast()); }
    @Test void testAll() { assertEquals("one two three ", l.dumpAll()); }
    @Test void testSingl() {
        Logs l2 = Logs.getInstance();
        assertSame(l, l2);
        l2.record("uno"); l2.record("due");
        assertEquals("due", l.dumpLast());
    }
}
```

La variante eager di `Fib` rende visibile la stessa condivisione tramite un indice:

```
final class Fib {
    private static final Fib obj = new Fib();
    private final int[] x = {1,2,3,5,8,13,21,34,55,89,144};
    private int i = 3;
    private Fib() { }
    static Fib getInstance() { return obj; }
    int getValue() {
        if (i < x.length) i++;
        return x[i - 1];
    }
    void revert() { i = 0; }
}
```

Con due riferimenti `f` e `f2`, due chiamate consecutive sul primo restituiscono 5 e 8; le due successive sul secondo restituiscono 13 e 21. Le righe stampate sono quindi `f 5 8` e `f2 13 21`.

Raggiunto l'ultimo elemento si continua a restituire 144; `revert` riporta l'indice a zero e la chiamata successiva restituisce 1. L'indice non appartiene alle variabili locali dei client.

Esercizio: Pagamenti unico e Pagamenti per cartella

Per trasformare la classe `Pagamenti` in Singleton si aggiungono, al suo interno, i seguenti membri e si mantengono le operazioni già definite:

```
private static final Pagamenti unico = new Pagamenti();
private Pagamenti() { }
public static Pagamenti getInstance() { return unico; }
```

Il client usa `Pagamenti.getInstance()` invece di `new Pagamenti()`. I file letti da client diversi confluiscono nello stesso stato; `svuota` agisce su tutti gli utilizzatori. L'unicità è quindi appropriata solo se quel comportamento è richiesto.

Una variante alternativa associa le istanze a chiavi di cartella; sostituisce il riferimento unico con una mappa e conserva il costruttore privato:

```
private static final Map<String, Pagamenti> istanze = new HashMap<>();
private Pagamenti() { }
public static synchronized Pagamenti getInstance(String cartella) {
    Pagamenti p = istanze.get(cartella);
    if (p == null) {
        p = new Pagamenti();
        istanze.put(cartella, p);
    }
    return p;
}
```

La stessa chiave restituisce la stessa istanza; chiavi diverse danno raccolte indipendenti. Il dominio delle chiavi determina quante istanze sono possibili: questa mappa non impone da sola una capacità massima prestabilita. Per un Multiton limitato occorre limitare le chiavi ammesse o rifiutare le nuove richieste al raggiungimento della capacità. Va anche stabilito se nomi diversi della stessa cartella debbano essere normalizzati. La sincronizzazione dell'accesso protegge la mappa, mentre gli aggiornamenti degli importi richiedono una valutazione distinta.

7.4 Factory Method: delegare la scelta del prodotto

L'intento di *Factory Method* è definire un'interfaccia per creare un oggetto lasciando alle sottoclassi la scelta della classe da istanziare. Un *framework*, nel senso usato qui, è una struttura di classi astratte che definisce lo scheletro di un'applicazione e lascia dettagli a chi lo specializza. Il framework deve restare generico e quindi conosce il tipo astratto `Product`, che non può istanziare, ma deve produrre oggetti concreti: il factory method rinvia questa decisione al `ConcreteCreator` mediante dispatch dinamico o *late binding*, cioè la scelta a runtime della ridefinizione del metodo in base al tipo dinamico dell'oggetto.

I ruoli sono:

Product l'interfaccia comune degli oggetti creati;

ConcreteProduct una sua implementazione;

Creator la classe, spesso astratta, che dichiara `factoryMethod()` con risultato di tipo `Product`; può anche fornire una creazione predefinita;

ConcreteCreator la sottoclasse che implementa o ridefinisce il factory method e istanzia il prodotto concreto.

Un'operazione concreta di `Creator` può chiamare `factoryMethod()` su `this`: se l'oggetto dinamico è un `ConcreteCreator`, viene eseguito il suo override, che restituisce un `ConcreteProduct`; il client conserva il risultato come `Product` e invoca il contratto comune. Tipicamente vi è una corrispondenza uno-a-uno fra concrete creator e concrete product, ma un creator può selezionare più prodotti con logica interna. Il vantaggio è l'intercambiabilità dei prodotti e l'estensione senza modificare i client che conoscono il contratto; il prezzo è la proliferazione di classi. Con un solo prodotto concreto e nessuna variazione prevista, la struttura può essere sovradimensionata.

Sono possibili varianti semplificate: creator e concrete creator nella stessa classe, factory statico e parametri che suggeriscono il prodotto. Queste varianti centralizzano comunque la creazione, ma il Factory Method classico è quello basato sulla delega alle sottoclassi; non si devono confondere le due descrizioni.

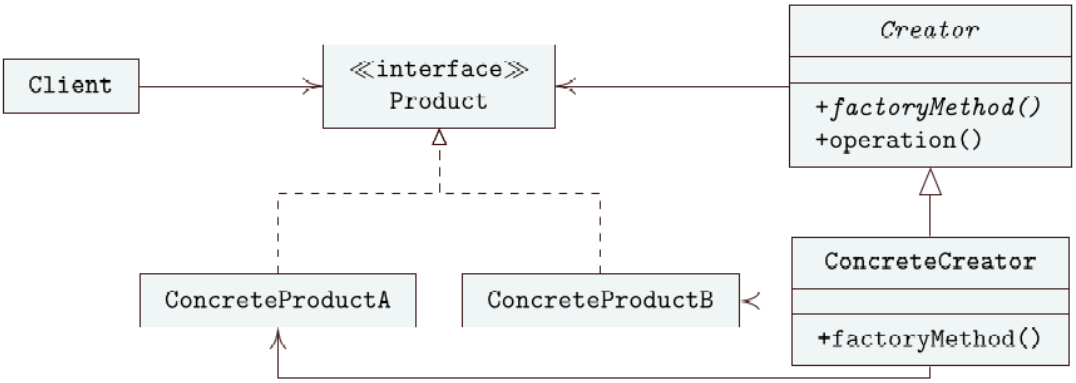


Figura 7.2: Ruoli del Factory Method nel diagramma delle classi.

Il Client richiede e usa il prodotto. Renderlo esplicito nel diagramma chiarisce la collaborazione senza imporre una classe con quel nome: il ruolo può essere svolto da qualunque parte del programma che utilizzi il servizio del creator.

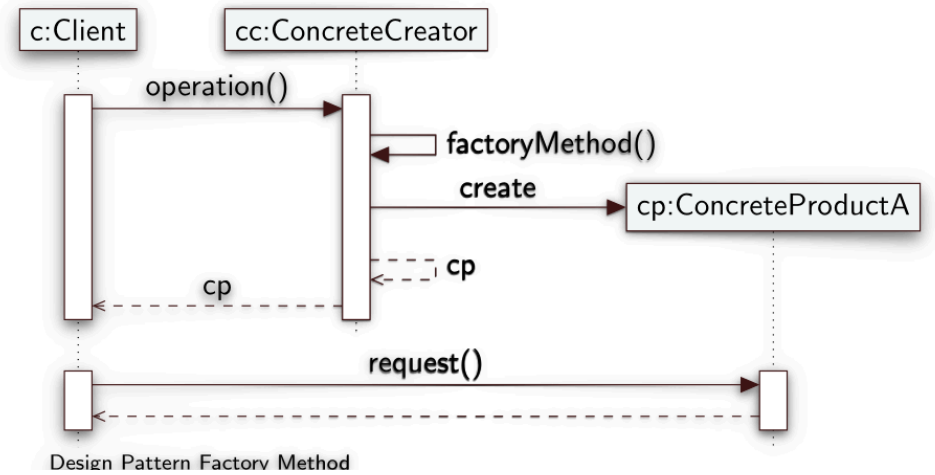


Figura 7.3: Creazione delegata al ConcreteCreator e uso del prodotto.

Il client invoca `operation()` sul creator; il metodo ereditato richiama il factory method sulla stessa istanza; il dispatch seleziona l'override del ConcreteCreator; il prodotto viene restituito e usato attraverso Product. Il diagramma delle classi può evidenziare soltanto il disaccoppiamento essenziale e lasciare implicita una dipendenza di configurazione presente nella

sequenza. Per ricostruire le dipendenze occorre quindi distinguere il collegamento necessario all'uso da quello necessario alla configurazione.

La scelta di rappresentare Product come interfaccia e Creator come classe astratta ha una motivazione specifica. Il prodotto può richiedere soltanto un contratto, senza stato comune; il creator può avere un contatore delle istanze, un metodo concreto `final` che lo aggiorna e un factory method astratto che crea il prodotto. Un'interfaccia non fornisce campi d'istanza né metodi d'istanza finali. Non è però un vincolo universale del pattern: il ruolo Product può anche essere una classe astratta se il dominio richiede stato e comportamento comuni.

7.4.1 Creator con contatore e sospensione degli studenti

Questo esempio usa `Studente` come interfaccia Product, distinta dalla gerarchia anagrafica. `StudenteAttivo` registra esami; `StBloccato` conserva la media al momento della sospensione e rifiuta nuovi esami.

```
import java.util.*;

interface Studente {
    boolean nuovoEsame(String materia, int voto);
    float getMedia();
}
record Esame(String materia, int voto) { }
class StudenteAttivo implements Studente {
    private final List<Esame> esami = new ArrayList<>();
    public boolean nuovoEsame(String m, int v) {
        esami.add(new Esame(m, v));
        return true;
    }
    public float getMedia() {
        if (esami.isEmpty()) return 0;
        float somma = 0;
        for (Esame e : esami) somma += e.voto();
        return somma / esami.size();
    }
}
class StBloccato implements Studente {
    private final float media;
    StBloccato(float media) { this.media = media; }
    public boolean nuovoEsame(String m, int v) { return false; }
    public float getMedia() { return media; }
}
abstract class Creator {
    private int numIstanze;
    public final Studente creaStudente() {
        numIstanze++;
        return getStudente();
    }
    public int getNum() { return numIstanze; }
    public abstract Studente getStudente();
    public abstract Studente sospendi(Studente s);
    public abstract Studente ripristina(Studente s);
}
class StCreator extends Creator {
    private boolean attivo = true;
    private final Map<Studente, Studente> bloccati = new HashMap<>();
    public Studente getStudente() {
```

```
        if (attivo) return new StudenteAttivo();
        return new StBloccato(0);
    }
    public Studente sospendi(Studente s) {
        Studente sospeso = new StBloccato(s.getMedia());
        bloccati.put(sospeso, s);
        return sospeso;
    }
    public Studente ripristina(Studente s) {
        return bloccati.remove(s);
    }
}
```

La sequenza è `client.creaStudente` sul creator, chiamata interna a `getStudente`, costruzione di `StudenteAttivo`, restituzione come `Studente` e invocazione di `nuovoEsame`. Il metodo finale impedisce di saltare il conteggio ridefinendolo. Il contatore registra però le chiamate a `creaStudente`: chiamare direttamente `getStudente` o costruire un bloccato in `sospendi` non lo incrementa.

La mappa associa il sostituto bloccato all'originale: `s = creator.sospendi(s)` cambia il riferimento del client; `s = creator.ripristina(s)` recupera gli esami originali e rimuove l'associazione. Un riferimento estraneo restituisce `null`. Altri client che conservano il riferimento allo studente attivo non vengono automaticamente bloccati. Il flag `attivo`, inizializzato a vero e privo qui di un'operazione di modifica, seleziona sempre lo studente attivo; l'eventuale configurazione alternativa richiede di completarne il contratto.

Lo stesso schema vale per un framework di documenti: `createDocument()` restituisce il tipo astratto `Document`, mentre creator specializzati producono `PdfDocument` o `WordDocument`. Il codice che apre, conserva e usa il documento resta dipendente dal contratto comune.

7.4.2 Posti del teatro: client, creatore e prodotto

`Posto` è Product; `PostoPalco` e `PostoPlatea` sono ConcreteProduct; `Posizioni` è Creator; `CreatorPosti` è ConcreteCreator. `Main` e `Biglietto` usano i posti attraverso il contratto comune.

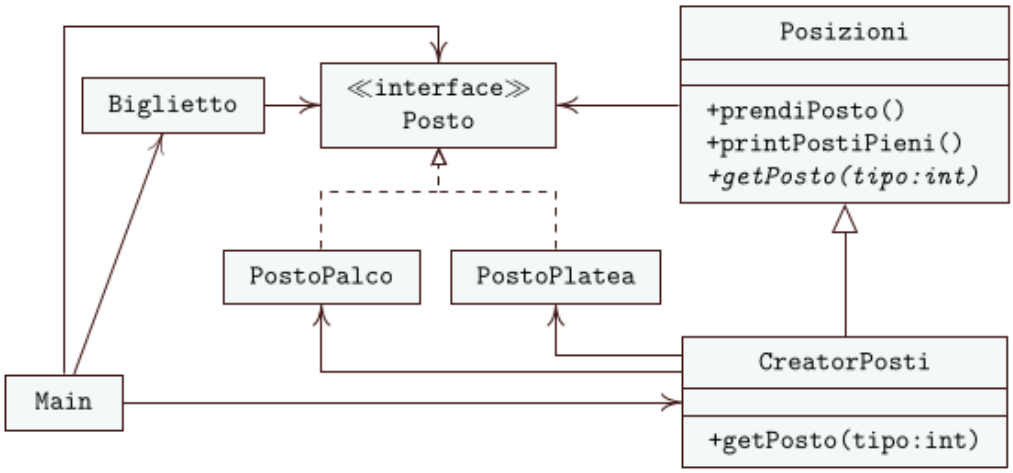


Figura 7.4: Nel teatro il Main configura il creatore e usa Posto; Biglietto conserva il posto assegnato.

Nel teatro, `Main` chiede un posto al creator, lo tratta come `Posto` e lo inserisce in un `Biglietto`; il biglietto dipende anch'esso dal contratto Posto. La variante che nomina `CreatorPosti` nel Main conserva una dipendenza concreta nel punto di configurazione. Dichiarare il riferimento come `Posizioni` disaccoppia l'uso; ricevere il creator già costruito nel costruttore disaccoppia anche la scelta del creatore dal client. Il parametro numerico di `prendiPosto(tipo)` sostituisce la conoscenza dei nomi `PostoPalco` e `PostoPlatea` con una convenzione da documentare.

Assegnazione, biglietto e condizioni di terminazione

`Posto` offre `getPosizione(): String`, `getCosto(): int` e `getSettore(): String`. Un `PostoPalco` sceglie nel costruttore un numero casuale fra 1 e 20; la posizione è quel numero convertito in testo. Per numeri da 1 a 10 il costo è 40 e il settore Blu; da 11 a 19 il costo è 50 e il settore Verde; il numero 20 costa 50 ed è Centrale.

`Posizioni` conserva un insieme ordinato di identificativi occupati e una lista dei posti assegnati, con capacità dichiarata 100. La logica di richiesta è:

```
public Posto prendiPosto(int tipo) {
    if (postiPieni.size() == 100) return null;
    Posto p;
    do { p = getPosto(tipo); }
    while (postiPieni.contains(p.getPosizione()));
    postiPieni.add(p.getPosizione());
    posti.add(p);
    return p;
}
public abstract Posto getPosto(int tipo);
```

`CreatorPosti.getPosto` restituisce `new PostoPalco()` per `tipo==1` e `new PostoPlatea()` negli altri casi. Il framework decide quando chiedere un prodotto e come registrarlo; la sottoclasse decide quale concreta costruire. `printPostiPieni` percorre la lista e stampa posizione, settore e costo.

Il client chiede prima `prendiPosto`, controlla che non sia `null`, poi costruisce `Biglietto(nome, posto)`. Il biglietto conserva il contratto `Posto`; `stampaDettagli` ne richiama posizione, settore e costo. Il messaggio finale riunisce intestatario e dati del posto senza riconoscerne la concreta.

La capacità globale 100 non garantisce che il ciclo termini: dopo aver occupato tutte le 20 posizioni di palco, nuove estrazioni di palco continuerebbero a produrre duplicati anche con meno di 100 posti totali. Occorre quindi distinguere disponibilità per categoria e capacità globale. Inoltre gli identificativi devono distinguere posti di settori diversi se i numeri coincidono. Senza una specifica completa della platea e dell'esaurimento per categoria non è possibile dedurre una politica unica: il frammento illustra Factory Method sotto la precondizione che esista ancora un posto ottenibile con identificativo nuovo; un gestore completo deve rendere esplicita e controllare tale precondizione.

7.5 Object Pool: acquisizione, rilascio e riuso

Un *Object Pool* è un deposito di istanze già create. Quando un client richiede un oggetto, il pool ne restituisce uno libero; alla fine dell'uso il client lo rilascia. Un pool a dimensione fissa non crea un oggetto se nessuno è disponibile; uno estendibile può crearne uno quando il deposito è vuoto. L'uso è esclusivo: un oggetto consegnato non deve restare disponibile nel pool. Prima del riuso può essere necessario reimpostarne lo stato, altrimenti il nuovo client vede dati del precedente. Un ConcreteCreator di Factory Method può implementare

questa politica senza cambiare l'interfaccia di richiesta; il pool stesso può essere unico e quindi combinato con Singleton.

Un pool può precaricare gli oggetti nel costruttore oppure crearli su richiesta. In un pool limitato, il numero di istanze create complessivamente va distinto dal numero di istanze attualmente libere: la sola dimensione della lista dei liberi non indica quante siano in uso. Quando si raggiunge la capacità, la politica di indisponibilità deve essere esplicita; non va introdotta una nuova istanza violando il limite.

Un `CreatorPool` estendibile rimuove un oggetto libero con `remove(0)` e, se il deposito è vuoto, costruisce un cerchio: rappresenta un pool estendibile, non un pool a capacità fissa. `releaseShape(s)` restituisce la risorsa. Occorre evitare rilasci duplicati, oggetti estranei e consegna simultanea dello stesso oggetto a due client. Prima di consegnare una risorsa riusata si ripristina lo stato previsto. La combinazione con Singleton controlla l'unicità del gestore del pool, non il numero delle risorse gestite.

Il codice rende visibile la politica: se la lista non è vuota riusa una `Shape`, altrimenti crea un `Circle`; `releaseShape` restituisce l'oggetto.

```
public class CreatorPool {
    private List<Shape> pool = new ArrayList<>();
    public Shape getShape() {
        if (pool.size() > 0) return pool.remove(0);
        return new Circle();
    }
    public void releaseShape(Shape s) { pool.add(s); }
}
```

In un pool con stato significativo, il rilascio o la nuova acquisizione deve aggiungere il ripristino, o *reset*, delle condizioni previste per un nuovo utilizzatore.

7.6 Dependency Injection: costruzione e wiring

La *Dependency Injection* (DI) è la tecnica per cui un oggetto riceve dall'esterno gli oggetti da cui dipende, anziché costruirli. Una *dipendenza* è un oggetto o servizio necessario al funzionamento di un altro oggetto. Nella *constructor injection* la dipendenza è parametro del costruttore: la costruzione è separata dall'uso e l'oggetto può essere completo dal primo istante. La *setter injection* configura invece la dipendenza dopo la costruzione: richiede di gestire uno stato inizialmente incompleto e il rischio di dimenticare il setter. L'*injector*, o *composition root*, crea e collega le istanze (*wiring*) in un punto. Poiché il factory method è un punto di creazione, può essere un posto naturale per iniettare le dipendenze nel prodotto; anche il creator scelto può essere iniettato nel client con tipo statico astratto.

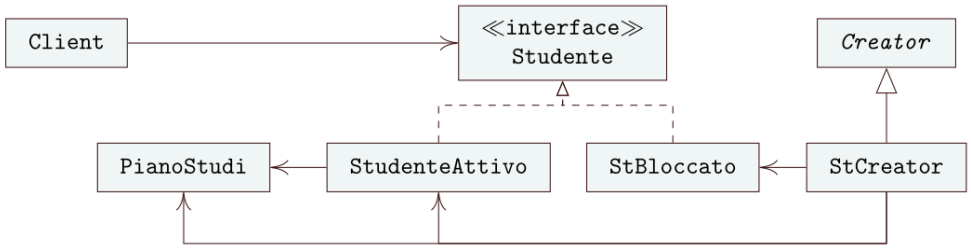


Figura 7.5: `StCreator` costruisce `StudenteAttivo` e gli fornisce `PianoStudi`.

La dipendenza deve essere disponibile quando l'oggetto inizia a essere usato. Un parametro nel costruttore rende esplicito il requisito e può essere conservato in un campo finale. Non

garantisce da solo un valore non nullo: tale invariante richiede un controllo o un contratto effettivamente rispettato. Per esempio:

```
final class StudenteAttivo {
    private final PianoStudi piano;
    StudenteAttivo(PianoStudi piano) {
        this.piano = java.util.Objects.requireNonNull(piano);
    }
}
```

Nell'esempio, il factory method realizza il wiring del prodotto. Anche il Main può fungere da composition root, creando creator, prototipi e client e collegandoli. Non ogni dipendenza dell'intera applicazione deve essere fabbricata da un'unica factory di prodotti: la responsabilità della factory deve rimanere coerente con il proprio dominio.

7.7 Abstract Factory: prodotti coerenti di una famiglia

L'intento di *Abstract Factory* è fornire un'interfaccia per creare famiglie di oggetti correlati senza specificarne le classi concrete. Una *famiglia* raccoglie prodotti di tipi diversi ma appartenenti alla stessa variante del sistema; per esempio un pulsante e una barra di scorrimento nello stesso look-and-feel. Il problema richiede sia di cambiare l'intera famiglia, sia di evitare di mescolare prodotti di famiglie diverse, che produrrebbero un'interfaccia incoerente.

`AbstractFactory` dichiara un'operazione di creazione per ciascun tipo di prodotto. Ogni `ConcreteFactory` implementa tali operazioni e produce esclusivamente la propria famiglia. Per ogni tipo di prodotto esiste un `AbstractProduct`, il contratto che nasconde le implementazioni; i concrete product lo implementano. Il client riceve una factory attraverso il tipo `AbstractFactory` e usa solo essa e gli `AbstractProduct`. In una rappresentazione a griglia, le righe sono i tipi di prodotto e le colonne le famiglie: una concrete factory seleziona una colonna intera.

A differenza del Factory Method, che disaccoppia dalla concreta di un prodotto, Abstract Factory disaccoppia da un'intera famiglia e ne preserva la coerenza. Le famiglie sono facilmente scambiabili; aggiungere un nuovo tipo di prodotto è invece oneroso, perché richiede un metodo nuovo nell'astrazione e in ogni factory concreta. L'Open/Closed Principle prescrive che un'entità software sia aperta all'estensione e chiusa alla modifica: qui si osserva nel poter aggiungere una famiglia attraverso una nuova factory, non nell'assenza assoluta di modifiche a tutte le astrazioni.

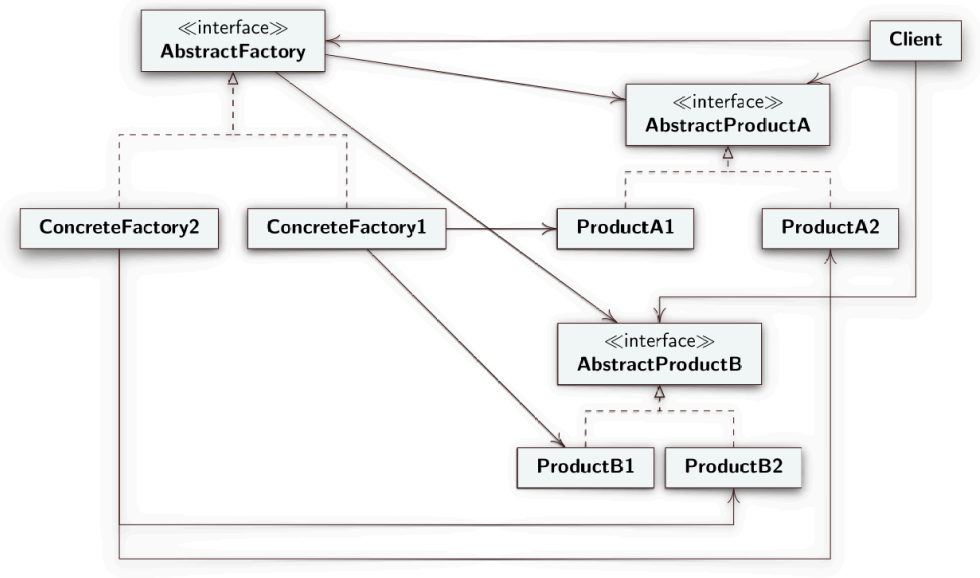


Figura 7.6: Due tipi di prodotto e due famiglie nell'Abstract Factory.

Le lettere A e B identificano tipi distinti di prodotto; i numeri 1 e 2 identificano le famiglie. Nell'esempio grafico, Button, Checkbox e Scrollbar sono tipi, Windows e macOS sono famiglie. Aggiungere Linux significa aggiungere una factory e i relativi prodotti; aggiungere un nuovo tipo di widget richiede invece di estendere il contratto di tutte le factory. Il client dipende direttamente dai contratti dei prodotti perché ne usa le operazioni, oltre che da AbstractFactory perché le richiede.

La coerenza è ottenuta facendo produrre alla factory selezionata soltanto oggetti della famiglia corretta e usando tale factory con disciplina. Il diagramma non impedisce magicamente a codice esterno di mescolare istanze ottenute da factory diverse. Ricorrere a cast ai prodotti concreti nel client indebolisce l'astrazione e rende meno agevole cambiare famiglia.

Per una Abstract Factory con m tipi di prodotto e n famiglie, la griglia concettuale ha $m \cdot n$ concrete product; una factory seleziona una delle n colonne. Questa relazione descrive la classificazione dei partecipanti, non il costo di esecuzione:

$$N_{\text{prodotti}} = mn, \tag{7.2}$$

ove N_{prodotti} è il numero di combinazioni concrete, m il numero di tipi e n il numero di famiglie.

7.7.1 Esempio: famiglie di icone e testi

Nell'esempio di Abstract Factory, `Creator` dichiara `getIcon()` e `getText()`; `CreatorEng` restituisce `Box` e `English`, mentre `CreatorJ` restituisce `Circle` e `Japanese`. Il client conserva la factory come `Creator`, ottiene un'icona e un testo dalla medesima famiglia e può cambiare variante cambiando la factory ricevuta.

```
interface Icon {
    void sad(); void happy(); void surprise();
}
class Circle implements Icon {
    public void sad() { System.out.print(" (> _ <) "); }
    public void happy() { System.out.print(" (^ _ ^) "); }
    public void surprise() { System.out.print(" (o _ o) "); }
}
```

```
class Box implements Icon {
    public void sad() { System.out.print(":-[ "); }
    public void happy() { System.out.print(":-] "); }
    public void surprise() { System.out.print(":-O "); }
}
interface Text {
    void welcome(); void attention(); void surprise();
}
class English implements Text {
    public void welcome() {
        System.out.println("::: Welcome. Nice to meet you. :::::");
    }
    public void attention() {
        System.out.println("::: Be quiet please! :::::");
    }
    public void surprise() { System.out.println("::: Oh, my dear! :::::"); }
}
class Japanese implements Text {
    public void welcome() {
        System.out.println("Youkoso. Konnichiwa! Hajimemashite.");
    }
    public void attention() {
        System.out.println("Shizuka ni shite kudasai!");
    }
    public void surprise() { System.out.println("Bikkuri shita!"); }
}
interface Creator {
    Icon getIcon(); Text getText();
}
class CreatorEng implements Creator {
    public Icon getIcon() { return new Box(); }
    public Text getText() { return new English(); }
}
class CreatorJ implements Creator {
    public Icon getIcon() { return new Circle(); }
    public Text getText() { return new Japanese(); }
}
```

Il client richiede i due prodotti e ne usa le operazioni:

```
Icon ic = c.getIcon();
Text t = c.getText();
ic.happy(); t.welcome();
ic.surprise(); t.surprise();
ic.sad(); t.attention();
```

Con `CreatorEng` ottiene le faccine Box seguite dai testi inglesi; sostituendo soltanto la factory con `CreatorJ` ottiene Circle e testi giapponesi. I due assi sono concreti: non si cambia un'icona isolata, ma la famiglia di oggetti richiesta dall'applicazione.

Nei diagrammi una freccia dalla concreta factory al prodotto astratto può essere omessa per non duplicare il contratto già dichiarato dall'interfaccia factory. L'omissione grafica non significa che il metodo concreto possa ignorare il tipo di ritorno. Analogamente, la dipendenza del client dai prodotti è diretta perché il client ne invoca i metodi: non si ricava una generale transitività di tutte le relazioni UML.

Nel toolkit grafico, `WindowsButton` e `MacScrollBar` appartengono a famiglie diverse. Mescolarli vanifica la coerenza che la selezione di una ConcreteFactory deve preservare. Una

nuova famiglia richiede i prodotti della nuova colonna; un nuovo tipo di prodotto richiede una nuova operazione in ciascuna factory.

7.8 Builder: costruire passo per passo un oggetto complesso

L'intento di *Builder* è separare la costruzione di un oggetto complesso dalla sua rappresentazione, affinché lo stesso processo costruttivo possa produrre rappresentazioni diverse. È applicabile quando l'algoritmo di costruzione deve restare indipendente dalle parti dell'oggetto e dal loro assemblaggio. Un caso frequente è un oggetto con molti attributi, alcuni opzionali, che devono diventare immutabili, cioè non modificabili dopo la costruzione.

Il costruttore *telescopico* offre molti costruttori con un numero crescente di parametri: consente di omettere valori, ma rende difficile al chiamante ricordare ordine e significato degli argomenti. I setter separati evitano la lunga lista di parametri ma impediscono agli attributi di essere `final` e non permettono di verificare nel costruttore che lo stato sia già consistente. Con Builder il client: (1) crea il builder con i dati richiesti; (2) configura passo per passo i dati opzionali con metodi che possono restituire il builder stesso; (3) invoca `build()` o l'equivalente, che verifica i dati necessari e crea il product immutabile.

I partecipanti sono `Builder`, interfaccia per costruire le parti; `ConcreteBuilder`, che assembla e conserva la rappresentazione in costruzione e fornisce il product; `Director`, che costruisce usando l'interfaccia Builder; e `Product`, l'oggetto complesso risultante. Il Director è un ruolo utile, non un obbligo del semplice builder fluente mostrato nell'esempio. Builder nasconde la struttura del product, isola costruzione e rappresentazione e dà controllo fine, passo dopo passo, fino alla consegna dell'oggetto completo.

Per Builder, `BigliettoBuilder` conserva dati provvisori quali titolare, agente, data e prezzo; metodi come `titolare(...)` restituiscono il builder, mentre `getBiglietto()` controlla che il titolare sia presente prima di costruire `Biglietto`. Il Director `InitBiglietto` mostra la sequenza di configurazione e la consegna finale dell'oggetto. L'esempio evidenzia che la validazione deve avvenire prima della consegna; uno stato provvisorio rappresentato da `null` deve essere controllato esplicitamente, perché Builder non garantisce automaticamente che tutti i dati richiesti siano presenti.

7.8.1 Tre modi di configurare Biglietto

Nella variante telescopica i campi di costruzione sono `titolare`, `agente`, `data` e `prezzo`. Il costruttore completo riceve tutti i valori; quello senza prezzo delega con `this(t,a,d,100)`, quello senza data usa `LocalDateTime.now()` e quello con il solo titolare usa l'agente `"BBBSelling"`. La catena `this(...)` concentra l'assegnazione in un costruttore, ma moltiplica le combinazioni da ricordare.

La variante con costruttore vuoto sostituisce i campi finali con campi assegnabili e offre `setTitolare`, `setAgente`, `setData` e `setPrezzo`. Fra due setter l'oggetto può essere incompleto, e il chiamante può dimenticare una configurazione necessaria. Il builder raccoglie invece i dati prima di consegnare il prodotto:

```
import java.time.LocalDateTime;

class Biglietto {
    final String titolare, agente;
    final LocalDateTime data;
    final double prezzo;
    private double acconto;
    Biglietto(String t, String a, double p, LocalDateTime d) {
        titolare = t; agente = a; prezzo = p; data = d;
```

```
    }
}
interface BBuilder {
    BBuilder agente(String a);
    BBuilder titolare(String t);
    BBuilder sconto(double sc);
    BBuilder data(LocalDateTime d);
    Biglietto getBiglietto();
}
class BigliettoBuilder implements BBuilder {
    private String titolare, agente = "SingleSeller";
    private LocalDateTime data;
    private double prezzo = 50;
    public BigliettoBuilder agente(String a) {
        agente = agente + " " + a;
        return this;
    }
    public BigliettoBuilder titolare(String t) {
        titolare = t; return this;
    }
    public BigliettoBuilder data(LocalDateTime d) {
        data = d; return this;
    }
    public BigliettoBuilder sconto(double sc) {
        if (prezzo >= 40 && sc <= 10) prezzo = (1 - sc / 100) * prezzo;
        return this;
    }
    public Biglietto getBiglietto() {
        if (titolare == null) return null;
        return new Biglietto(titolare, agente, prezzo, data);
    }
}
class InitBiglietto {
    Biglietto getBiglietto() {
        return new BigliettoBuilder().agente("Tim Seller")
            .titolare("Mario Rossi").sconto(5)
            .data(LocalDateTime.now()).getBiglietto();
    }
}
```

`InitBiglietto` svolge il ruolo Director: stabilisce l'ordine dei passi e ottiene il prodotto. La catena produce agente `"SingleSeller Tim Seller"` e prezzo $50(1 - 5/100) = 47{,}5$. Il controllo dello sconto richiede contemporaneamente prezzo corrente almeno 40 e percentuale al massimo 10; non impone un limite inferiore alla percentuale e non verifica tutti i dati del prodotto. Se il dominio esclude percentuali negative o date mancanti occorre un vincolo aggiuntivo, che non va dato per implicito. Due sconti consecutivi agiscono sul prezzo già ridotto. `getBiglietto` rifiuta soltanto il titolare nullo, restituendo `null`.

I quattro campi di costruzione sono finali, ma `acconto` è distinto e mutabile: se il contratto permette di aggiornarlo, il biglietto nel suo complesso non è immutabile. Builder controlla la costruzione; l'immutabilità successiva dipende da tutti i campi e dalle operazioni esposte.

7.9 Prototype: creazione mediante copia

L'intento di *Prototype* è specificare i tipi degli oggetti da creare mediante un'istanza prototipo e creare nuovi oggetti copiandola. È utile quando lo stato iniziale è complesso e simile a

quello di un oggetto già configurato, oppure quando scrivere una concreta nell'espressione `new` accoppierebbe il client a quella classe. Il client usa `prototipo.clone()` attraverso il contratto `Prototype`: il tipo emerge dall'istanza, non dal nome della concreta scritto nel client.

`Prototype` dichiara la clonazione; `ConcretePrototype` la realizza; il client crea copie senza nominare la concreta. Un `PrototypeRegistry` è un registro separato, tipicamente una mappa da chiave semantica a prototipo: permette di registrare o rimuovere prototipi a runtime. Il registro può essere Singleton o una dipendenza iniettata. Così, per introdurre una variante non occorre necessariamente una gerarchia di creator: si può registrare un nuovo oggetto prototipo. Una composizione, per esempio un pannello con controlli, può essere clonata come struttura intera.

La semantica della copia deve essere esplicita. Una *copia superficiale* duplica l'oggetto ma conserva i riferimenti agli oggetti raggiunti dai suoi campi. Una *copia profonda* duplica anche gli oggetti mutabili che devono appartenere allo stato indipendente del clone. La scelta dipende dal contratto: risorse condivise e oggetti immutabili possono restare comuni.

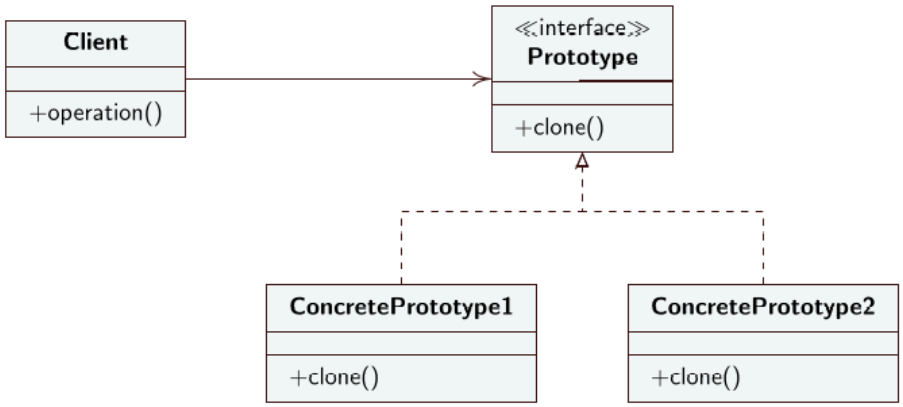


Figura 7.7: Il client chiede la copia tramite l'interfaccia Prototype.

Il registro contiene prototipi già esistenti: qualcuno deve crearli e configurarli prima della prima clonazione. Una nuova variante di stato, per esempio un cerchio rosso rispetto a uno blu, può richiedere soltanto una diversa istanza registrata; un nuovo tipo strutturale, come un triangolo con attributi propri, richiede comunque la sua implementazione di ConcretePrototype. Prototype evita la corrispondente gerarchia di creator, non elimina le classi necessarie al dominio.

`addAll()` inserisce gli elementi in una collezione di destinazione senza clonare gli oggetti referenziati. Una lista nuova evita di condividere il contenitore, ma può ancora condividere tutti gli elementi mutabili. Per copiare un grafo con cicli occorre memorizzare in una mappa ogni coppia originale–clone prima di proseguire sui riferimenti: incontrando di nuovo l'originale, si riusa il clone già creato. Si evita così la ricorsione indefinita e si preservano le relazioni di condivisione previste.

La copia può essere seguita da un'inizializzazione specifica: un contratto usato come prototipo può conservare identificativo, data e cliente del modello, che devono essere sostituiti per il nuovo caso. Copiare l'oggetto non decide automaticamente quali dati debbano cambiare.

Rispetto alle classi `CerchioCreator`, `QuadratoCreator` e `TriangoloCreator`, un registro di prototipi evita una gerarchia parallela di creatori. Le varianti `CerchioRosso`, `CerchioBlu` e `CerchioGrande` possono essere sostituite da stati diversi di uno stesso tipo Cerchio. Un pannello composto da Button, Label e TextField mostra invece una copia di struttura: si deve decidere quali parti clonare e quali mantenere condivise.

7.9.1 Esempio: agenda di impegni e quaderno di note

Una nuova agenda può partire dagli impegni già inseriti, oppure un quaderno dalle note precedenti. Se originale e clone condividono la stessa collezione, aggiungere o rimuovere un elemento tramite uno modifica anche ciò che vede l'altro. Se la collezione è copiata ma gli elementi sono condivisi, cambiano indipendentemente gli elenchi, mentre modificare un impegno mutabile resta visibile a entrambi. Una copia profonda separa anche gli elementi per i quali il contratto richiede indipendenza.

Un Documento con font, margini, lingua e stile già configurati mediante numerose operazioni è un buon candidato a prototipo: copiarne lo stato evita di ripetere la stessa preparazione, poi si modificano i dati specifici del nuovo documento. Nel wiring di agenda e note, il Main può creare il `Calendario` e il `Notebook` iniziali e iniettarli nel client, che lavorerà con i loro contratti di copia. La prima istanza deve comunque essere costruita prima di poterla clonare.

Nel caso ciclico Dipendente–Manager, il dipendente riferisce il proprio manager e il manager conserva la lista dei dipendenti. Una copia ricorsiva ingenua passerebbe dal dipendente al manager e di nuovo allo stesso dipendente senza terminare. Si crea prima il clone vuoto del dipendente, lo si registra nella mappa originali–cloni, poi si copia il manager. Quando la copia della lista incontra il dipendente originario, recupera il clone già registrato: il manager clonato punta così allo stesso dipendente clonato, senza creare un duplicato. Per oggetti distinti ma uguali per contenuto la mappa deve distinguere l'identità degli originali, per esempio con `IdentityHashMap`.